

Interstate Traffic Forecasting : A Data-Centric SimpleRNN Implementation

CA6000 – Applied AI Programming

NTU CCDS MCAAI

Haocheng Wang
G2509275G

HWANG051@e.ntu.edu.sg

1. Introduction

This project **predicts next-hour traffic volume using the previous 24 hours of data**. A ***data-centric approach*** is adopted, where exploratory data analysis and data cleaning inform modeling decisions. A ***SimpleRNN*** is employed to model potential sequential dependencies, and its performance is compared against baseline models, including *Linear Regression* and a *Fully Connected Neural Network (FCNN)*.

The primary objective of this study is methodological and process-oriented, with an emphasis on time-series data preparation, principled data selection, and understanding how recurrent models process sequential information.

2. Dataset

2.1. Source & Import

Source: This project utilizes *Interstate Traffic Dataset (US)* from Kaggle.

Description: This dataset contains **hourly data on the traffic volume** for *westbound I-94*, a major interstate highway in the US that connects *Minneapolis* and *St Paul, Minnesota*. The data was collected by the *Minnesota Department of Transportation (MnDOT)* from 2012 to 2018 at a station roughly midway between the two cities.

Manual Inspection: After importing the dataset, an initial inspection was conducted using *df.info()*, *df.describe()*, and direct examination of the raw CSV file. Tables 1 and 2 provide an overview of the raw data.

Table 1. Raw Dataset Snapshot (First 5 & Last 5 Rows)

Index	date_time	traffic_volume (Target)	holiday	temp (K)	rain_1h (mm)	snow_1h (mm)	clouds_all (%)	weather_main	weather_description
0	02-10-2012 09:00	5545	NaN	288.28	0.0	0.0	40	Clouds	scattered clouds
1	02-10-2012 10:00	4516	NaN	289.36	0.0	0.0	75	Clouds	broken clouds
2	02-10-2012 11:00	4767	NaN	289.58	0.0	0.0	90	Clouds	overcast clouds
3	02-10-2012 12:00	5026	NaN	290.13	0.0	0.0	90	Clouds	overcast clouds
4	02-10-2012 13:00	4918	NaN	291.14	0.0	0.0	75	Clouds	broken clouds
...	...	...	...	...	...	...	...	...	...
48199	30-09-2018 19:00	3543	NaN	283.45	0.0	0.0	75	Clouds	broken clouds
48200	30-09-2018 20:00	2781	NaN	282.76	0.0	0.0	90	Clouds	overcast clouds
48201	30-09-2018 21:00	2159	NaN	282.73	0.0	0.0	90	Thunderstorm	proximity thunderstorm
48202	30-09-2018 22:00	1450	NaN	282.09	0.0	0.0	90	Clouds	overcast clouds
48203	30-09-2018 23:00	954	NaN	282.12	0.0	0.0	90	Clouds	overcast clouds
Dtype	(object)	(int64)	(object)	(float64)	(float64)	(float64)	(int64)	(object)	(object)

Table 2. Statistical Summary of Numerical Features

features	traffic_volume	temp	rain_1h	snow_1h	clouds_all
count	48204	48204	48204	48204	48204
mean	3259.818355	281.205870	0.334264	0.000222	49.362231
std	1986.860670	13.338232	44.789133	0.008168	39.015750
min	0.000000	<u>0.000000</u>	0.000000	0.000000	0.000000
25%	1193.000000	272.160000	0.000000	0.000000	1.000000
50%	3380.000000	282.450000	0.000000	0.000000	64.000000
75%	4933.000000	291.806000	0.000000	0.000000	90.000000
max	7280.000000	310.070000	<u>2831.300000</u>	0.510000	100.000000

This preliminary exploration suggested several potential but unverified data quality issues:

1. **Numerical validity:** *temp*, *rain_1h*, and *snow_1h* exhibited extreme values that may distort statistical analysis and model training.
2. **Feature sparsity:** *holiday*, *rain_1h*, *snow_1h*, and *clouds_all* contained a large proportion of zero or binary-valued entries.
3. **Temporal integrity:** the multi-year hourly time index required verification of temporal continuity and uniqueness.
4. **Categorical consistency:** *weather_main* and *weather_description* showed relatively high cardinality, raising concerns about encoding suitability.

2.2. Examine Potential Issues

The suspected issues were verified through a purely code-based diagnostic process using standard data inspection utilities from the pandas library. This step focused on confirming the relevance of previously identified concerns rather than introducing new ones.

Diagnostics were structured into four categories—numerical validity, feature sparsity, temporal integrity, and categorical consistency—and the results directly informed subsequent data cleaning and feature handling decisions.

Table 3. Consolidated Data Health Audit

Feature Name	Raw Type	Primary Diagnosis & Key Metrics	Implication for Pipeline
traffic_volume	int64	Clean. Target variable.	Keep. Use for Loss calculation.
date_time	object	Discontinuity: Script detected 11,976 missing hours and 7,629 duplicates.	Must Fix. Parse to datetime, deduplicate, and re-index to fix gaps.
holiday	object	Extreme Sparsity: 99.9% NaN. Script found missing years.	Impute or Drop. Treat NaN as "Non-Holiday"; regenerate features via library.
temp	float64	Physical Anomaly: 10 rows contain impossible 0 Kelvin readings.	Filter. Remove specific rows (Physical constraints).
rain_1h	float64	Extreme Outlier: Max value 9,831 mm is impossible.	Filter. Remove specific rows (Outliers).
snow_1h	float64	Suspicious Range: Max value 0.51mm; 99% zeros.	Scale or Drop. Requires robust normalization; low variance may limit utility.
clouds_all	int64	Clean. Valid percentage range [0-100].	Keep. Ready for scaling.
weather_main	object	Unreliable: Semantic conflicts (e.g., Rain text vs 0mm volume).	Candidate for Removal. Rely on numeric features (rain_1h) instead.
weather_desc	object	Redundant: High cardinality (38 categories) & low value.	Candidate for Removal. Will verify redundancy in EDA.

3. Exploratory Data Analysis (EDA)

3.1. Canonical Pattern

To validate temporal periodicity, a *canonical week* profile was constructed by aggregating the full dataset (≈ 286 weeks). Traffic volume was grouped into 168 hourly slots (7×24), allowing random fluctuations to be smoothed and the underlying weekly pattern to emerge. The mean traffic volume represents the typical weekly profile, while the standard deviation captures variability and is visualized as a shaded uncertainty band.

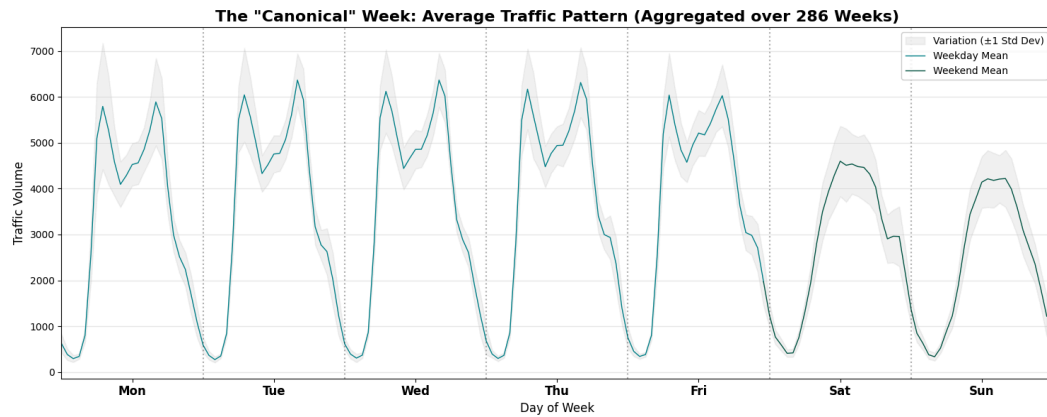


Fig.1. Canonical Average Week Pattern

3.2. Correlation Analysis

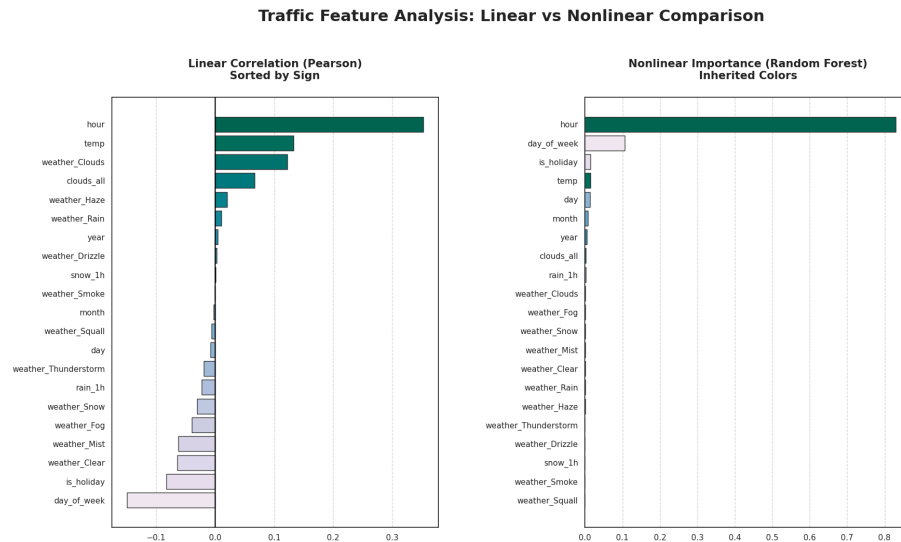


Fig.2. Correlation Analysis (Linear & Non-linear)

To investigate feature–traffic relationships, this study conducted two complementary correlation analyses. Prior to testing, a light preprocessing step was applied to ensure analytical validity, including the removal of previously identified anomalous values and the

extraction of temporal components (e.g., *hour*, *day of week*) from the timestamp to expose potential cyclic patterns.

Feature relevance was then examined using two methods: Pearson correlation to capture linear relationships and Random Forest feature importance to assess nonlinear contributions and relative influence magnitude.

As illustrated in Figure 2, temporal features—particularly *hour*—emerge as the dominant predictors, accounting for the majority of explanatory power. Both *day_of_week* and *is_holiday* show strong negative linear correlations, indicating reduced traffic during non-working periods. In contrast, weather-related features exhibit limited importance, suggesting that weather plays a secondary role compared to intrinsic temporal dynamics.

3.3. Consistency Check: Subjective Labels vs. Objective Sensors

To assess the reliability of categorical weather descriptions, a cross-validation analysis was conducted between the subjective *weather_main* labels (clouds, clear and others) and the objective *clouds_all* sensor readings (Figure 3.3).

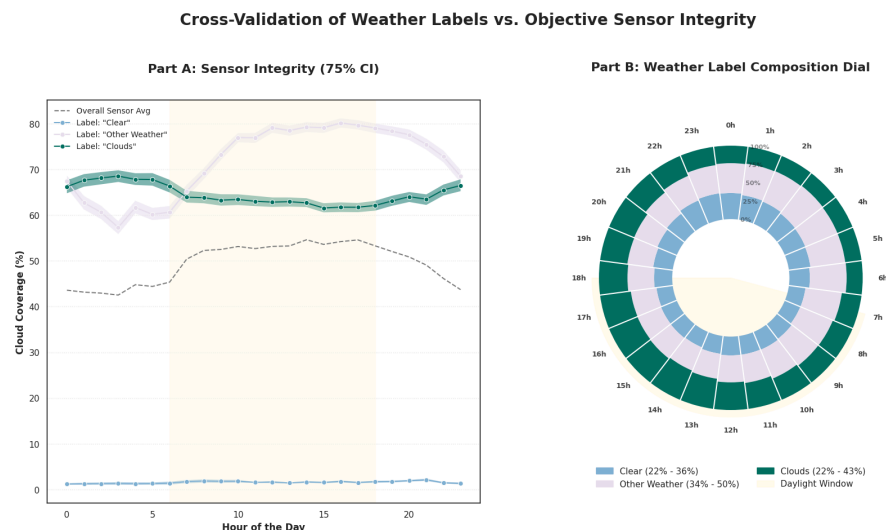


Fig.3. Consistency Analysis of Categorical Weather Labels and Sensor Metrics

Figure 3.A compares the overall sensor average with label-specific cloud coverage across the diurnal cycle. While the overall average cloud coverage increases noticeably during daylight hours, the mean sensor values within the *Clouds* category remain flat or slightly decrease, revealing an apparent inconsistency.

Figure 3.B explains this pattern by examining label frequency over time. The occurrence of *Clouds* labels rises during daylight hours, while *Clear* labels become rare. This indicates an observational bias: clouds are more likely to be visually detected and recorded during the day, whereas thin or partial cloud cover at night may be misclassified as *Clear* due to limited visibility.

Overall, the *weather_main* labels are shown to be subjective and time-dependent, reflecting human perception rather than purely physical conditions. As a result, the continuous *clouds_all* sensor feature provides a more robust and objective representation and is therefore prioritized for model training.

4. Methodology

This study follows a Data-Centric AI framework, structured into four sequential stages: Analysis & Diagnosis, Feature Construction, Sequence Generation, and Model Construction & Optimization (see Figure 4).

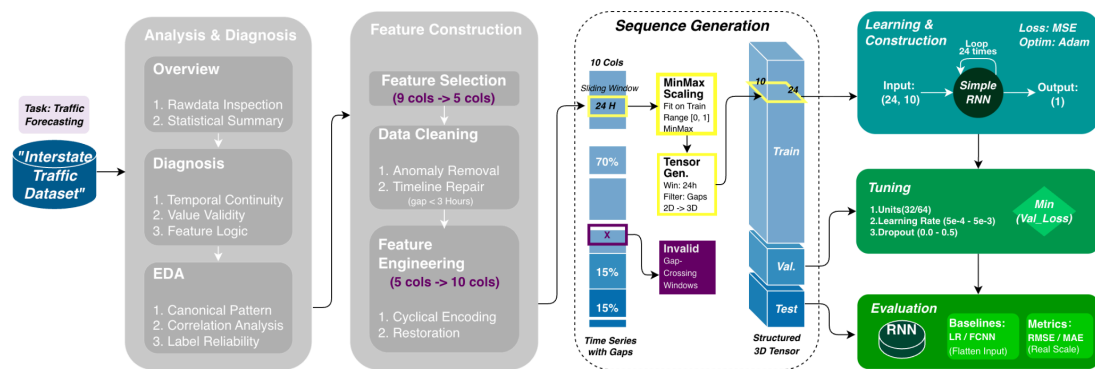


Fig.4. Schematic Overview of the Four-Stage Methodology

Stage 1 – Analysis & Diagnosis:

Raw data were inspected for anomalies, temporal gaps, and inconsistencies between categorical labels and sensor readings to ensure dataset integrity.

Stage 2 – Feature Construction:

Data cleaning and interpolation were applied; feature dimensionality was reduced via correlation analysis (Random Forest & Pearson). Temporal features were encoded cyclically, and the timeline was restored for continuity.

Stage 3 – Sequence Generation:

MinMax scaling normalized features. Input-output sequences ($X[t-23...t]$, $y[t+1]$) were created with a 24-hour sliding window, discarding any crossing gaps > 3 hours. Data were split into Train (70%), Validation (15%), and Test (15%).

Stage 4 – Model Construction & Optimization:

A SimpleRNN captured temporal dependencies; Linear Regression and FCNN served as baselines. MSE loss was optimized with Adam, guided by validation for hyperparameter tuning and early stopping. Performance was evaluated via RMSE and MAE on inverse-transformed data.

Implementation Stack:

Pandas/NumPy for data handling, Matplotlib and Seaborn for visualization, TensorFlow and Keras for deep learning, and Scikit-learn for preprocessing and evaluation.

5. Feature Construction

5.1 Feature Selection & Anomaly Removal

Guided by the "triangulation of evidence" (Figs. 2, 3 and Table 3), features were selected based on statistical relevance, data quality, and semantic integrity. This process ensured the removal of noise while preserving meaningful temporal and environmental signals.

The resulting streamlined dataset, stored as *df_minimal*, includes only the validated features summarized in the logic matrix below (Table 4). Key corrections included:

- **Anomaly Filtering:** Removed physical impossibilities, such as 0 K temperatures and extreme sensor-error rainfall values.
- **Reconstruction:** Replaced the defective raw holiday column with a library-generated binary `is_holiday` feature to accurately capture traffic patterns.
- **Bias Mitigation:** Dropped subjective human labels (`weather_main`) in favor of objective sensor data (`clouds_all`) to ensure data integrity.

Table 4. Feature Selection & Cleaning Logic Matrix

Feature Name	Action	Linear Signal (r)	Nonlinear Power (I)	Data Diagnosis	Decision Logic (Rationale)
<code>traffic_volume</code>	Target	N/A	N/A	Clean	Target Variable.
<code>date_time</code> (Parent Source)	Explode	N/A	Source of Truth	Discontinuity	Foundation for extracting temporal patterns.
↳ <code>hour</code>	Keep	Mixed	Rank #1 (>80%)	Clean	Dominant Driver. Primary predictor of daily volume.
↳ <code>day_of_week</code>	Keep	$r \ll 0$ (↓↓)	Rank #3 (≈5%)	Clean	Weekly Cycle. Captures Work vs. Weekend patterns.
↳ <code>month</code>	Keep	Mixed	Rank > 6	Clean	Seasonality. Captures long-term annual trends.
<code>temp</code>	Keep	$r > 0$ (↑)	Rank #2 (≈12%)	Physical Anomaly	High Signal. Key environmental influencer.
<code>holiday</code>	Regen.	$r \ll 0$ (↓↓)	Local I_High	Structural Defect	Event Trigger. Strongest switch for traffic reduction.
<code>clouds_all</code>	Keep	$r < 0$ (↓)	Rank ~5 (<2%)	Clean	Objective Proxy. More reliable than human labels.
<code>rain_1h</code>	Drop	$r \approx 0$	Rank >6 (<1%)	Outliers + Sparsity	No Information. High sparsity & low correlation yield no gain.
<code>snow_1h</code>	Drop	$r \approx 0$	Rank Last (≈0%)	Extreme Sparsity	Low Variance. Insufficient info for learning.
<code>weather_main</code>	Drop	Mixed	Redundant	Observational Bias	Unreliable. Conflicts with objective sensor data.
<code>weather_desc</code>	Drop	Weak	Redundant	High Cardinality	Noise. Too many categories dilute the signal.

5.2. Time Series Restoration & Feature Engineering

After feature selection, the working dataset (*df_minimal*) retained high-value predictors but exhibited structural issues, including duplicate timestamps and large temporal gaps. A structured preprocessing pipeline was therefore applied to ensure temporal validity for sequence modeling.

Structural Repair (`df_minimal` → `df_repaired`).

Duplicate hourly records were aggregated using the mean and reindexed to a strict hourly timeline, explicitly exposing missing timestamps. Linear interpolation was applied only to short gaps (≤ 3 hours) to preserve local continuity, while longer gaps were intentionally left as NaN to avoid introducing synthetic trends.

Temporal Feature Engineering (`df_repaired` → `df_features`).

From the repaired timestamp index, hour, day_of_week, and month were extracted and encoded using sine–cosine transformations:

- $\sin_x = \sin(2 * \pi * x / \text{Max})$
- $\cos_x = \cos(2 * \pi * x / \text{Max})$

This encoding was applied to daily (24), weekly (7), and annual (12) cycles, resulting in `df_features`, which combines restored data with continuous temporal context.

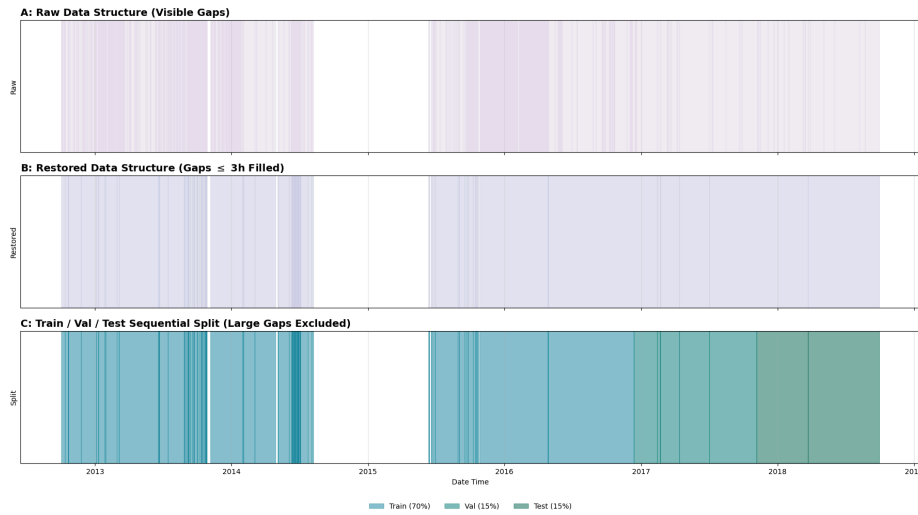


Fig.5. The Time Series Restoration and Gap-Aware Splitting

5.3. Logical Data Splitting (Gap-Aware)

To prevent temporal leakage, `df_features` was split chronologically into training (70%), validation (15%), and test (15%) sets using boolean masks.

A data point was assigned to a subset if and only if it lay within the corresponding time range **and** contained valid (non-NaN) sensor data. As a result, large gaps preserved during structural repair were excluded from all subsets by construction (Fig. 5).

5.4. Feature Scaling & Segment Identification

Before sequence generation, a gap-aware segmentation variable (`segment_id`) was introduced by cumulatively marking NaN regions in the target variable, where changes in `segment_id` indicate temporal discontinuities. The datasets were then physically extracted as `train_df`, `val_df`, and `test_df`.

All continuous features were normalized to the range $[0, 1]$ using MinMax scaling. To avoid data leakage, the scaler was fitted exclusively on **train_df** and applied to validation and test sets.

5.5. Gap-Safe Sliding Window Generation

Finally, the scaled datasets were transformed into supervised learning samples using a sliding window approach. Each input consists of the past 24 hours of multivariate observations, with traffic volume at the next time step as the target.

During window construction, **segment_id** was explicitly checked to ensure that no sequence crossed a preserved data gap. The resulting tensors (**X_train**, **y_train**, **X_val**, **y_val**, **X_test**, **y_test**) contain only temporally continuous sequences and are fully compatible with RNN-based models.

6. Implementation

The SimpleRNN was implemented in **TensorFlow/Keras** using **3D tensors** (Batch, 24, 10) as input. A systematic ablation study was conducted across eight configurations to optimize **units**, **dropout**, and **learning rate** (Table 5). Training employed the **Adam** optimizer, **MSE** loss, and a **batch size of 64**. To ensure optimal convergence, an **Early Stopping** callback (patience=20) was utilized to monitor validation loss and restore the best performing weights.

Table 5. Model Tuning & Experiment Results

Exp. ID	Configuration	Units	Dropout	Learning Rate	Best Val Loss
6	No Drop (Best)	64	0.0	0.001	0.00117
7	Light NoDrop	32	0.0	0.001	0.00121
4	Low LR	64	0.2	0.0005	0.00127
8	Mild Drop	64	0.1	0.001	0.00127
1	Baseline	64	0.2	0.001	0.00131
2	Light	32	0.2	0.001	0.00132
5	High Drop	64	0.5	0.001	0.00136
3	High LR	64	0.2	0.005	0.00147

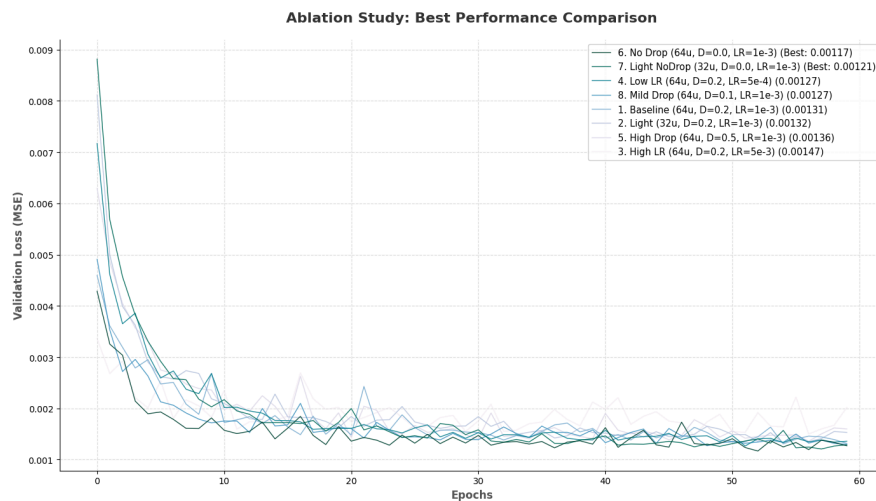


Fig.6. Training/Validation curves

Figure 6. shows that higher dropout slows convergence and reduces performance, while a large learning rate (0.005) leads to volatile loss curves. The optimal configuration—64 hidden units, no dropout, and LR = 0.001—achieved smooth, stable training and the best validation MSE (0.00117), indicating that the cleaned dataset has a high signal-to-noise ratio and that 64 units effectively capture the nonlinear traffic patterns without overfitting.

7. Results and Evaluation

The model was evaluated against two baselines using RMSE and MAE. As shown in Table 6, the SimpleRNN (64 units, 4,865 parameters) outperformed both Linear Regression and the FCNN, achieving the lowest errors (RMSE = 240.69 cars, MAE = 158.71 cars) despite a smaller parameter count, reducing RMSE by 35.6% compared to Linear Regression.

Table 6. Comparative Performance Metrics (RMSE & MAE)

Model Architecture	Input Shape	Structure	Total Parameters	RMSE (Cars)	MAE (Cars)	Error Reduction (vs. Baseline)
Linear Regression (Baseline)	(N, 240) Flattened	Input(240) → Dense(1)	241	373.72	251.23	—
FCNN (Multi-Layer Perceptron)	(N, 240) Flattened	Input(240) → Dense(32) → Dense(8) → Dense(1)	7,985	272.09	190.15	27.2%
SimpleRNN (Proposed)	(N, 24, 10) Sequence	Input(10) → SimpleRNN(64) → Dense(1)	4,865	240.69	158.71	35.6%

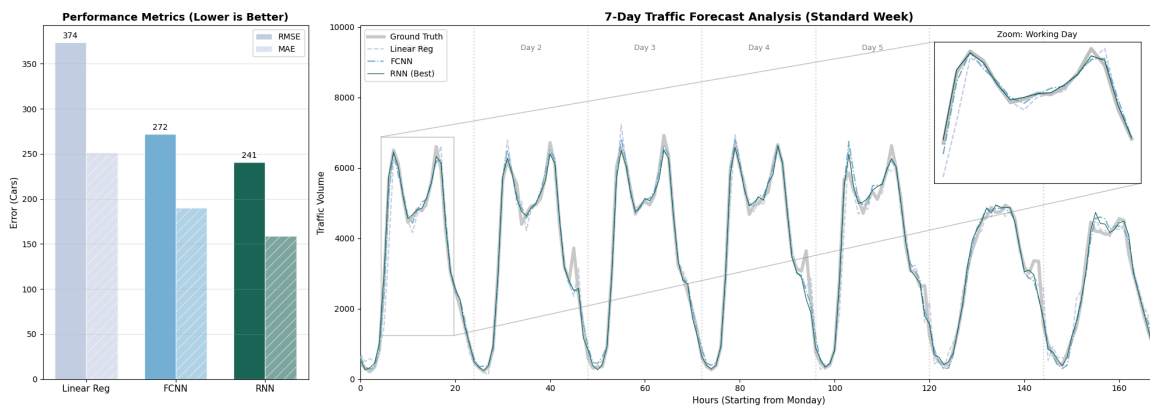


Fig.7. Comparative Performance Evaluation: Error Metrics (RMSE/MAE) and 7-Day Traffic Pattern Reconstruction.

Evaluation over a standard 7-day test window (Figure 7) demonstrates that the SimpleRNN closely tracks traffic patterns, capturing morning and evening rush hour peaks with higher fidelity than the baselines. Its predictions align well with the ground truth, showing stable temporal behavior, minimal lag during rapid transitions, and accurately reflecting daily and weekend traffic cycles.

8. Discussion & Insights

This assignment offered a full experience of inspecting, cleaning, and engineering a real traffic dataset. The SimpleRNN clearly showed its strength in handling temporal sequences, outperforming a flattened FCNN by ~11.5% RMSE. Notably, the best results came from the

No-Dropout configuration, confirming that careful cleaning and cyclical feature encoding effectively reduced noise.

Beyond the model itself, the end-to-end process—from detecting gaps, filling short missing intervals, to creating segment-aware sliding windows—provided practical insights into data preparation, boosting my confidence in tackling future time-series projects with cleaner, more robust datasets. Saving trained models is also crucial, which ensures subsequent analysis can be conducted reliably.

Future Outlook: While the current SimpleRNN is effective, future iterations will explore Transformer-based architectures and attention mechanisms for capturing longer-term dependencies, as well as Random Input Masking to enhance real-time resilience.

9. Conclusion

A SimpleRNN with 64 units, no dropout, and LR=0.001 achieved a validation MSE of 0.00117, outperforming linear and FCNN baselines, showing that sequential modeling and clean, well-engineered features enable accurate short-term traffic forecasts.

10. AI Assistance Summary

This project leveraged AI tools —Gemini and ChatGPT — across three main usage modes:

1. **Proactive Prompting:** I provided explicit instructions to the AI to perform tasks such as generating visualizations or processing data in a specified way. Due to the extensive prompting involved, **related instructions are documented at the beginning of each code block in the notebook.**
2. **Code Assistance:** When encountering errors, I submitted both the code and error messages to the AI, which returned corrected versions and explanations. Beyond debugging, I also used AI to optimize and extend existing code by providing the current code along with new requirements, receiving updated implementations that met the additional needs.
3. **Analysis & Insights:** When I asked the AI to analyze problems or generate code, it often introduced new ideas or techniques I hadn't considered—such as cyclical encoding for temporal features or using Python libraries like *holidays*—which provided valuable insights and inspired improvements to my workflow.

References :

- Tanwar, A. (2019). *Metro Interstate Traffic Volume [Data set]*. Kaggle. <https://www.kaggle.com/datasets/anshtanwar/metro-interstate-traffic-volume>
- TensorFlow Developers & Chollet, F. (2023). *TensorFlow and Keras: Large-scale machine learning on heterogeneous systems [Software]*. <https://www.tensorflow.org/>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). *Scikit-learn: Machine learning in Python*. *Journal of Machine Learning Research*, 12, 2825–2830. <https://scikit-learn.org/>
- Elman, J. L. (1990). *Finding structure in time*. *Cognitive Science*, 14(2), 179–211. https://doi.org/10.1207/s15516709cog1402_1